

CYLAB SECURITY ACADEMY

Challenge Write-Up Template

Challenge Information

Student Name	Evallo, Daniella Kim N.
Section / Class	BSIT-3G
Challenge Name	Password Profiler
Category	General Skills
Points / Difficulty	Easy
Date Solved	08/10/2026
Flag Format Given	PicoCTF{...}

1. Challenge Description

The challenge provided personal information about a target, a SHA-1 hash of the target's password, and a password-checking script. The goal was to use the supplied information to generate a custom wordlist with CUPP, find the password whose SHA-1 hash matched the supplied hash, and submit the resulting flag.

2. Initial Analysis / Reconnaissance

I first reviewed the challenge prompt, the downloadable files, and the revealed hint. The challenge was classified as General Skills and Easy. The hint identified CUPP as a Python tool for generating custom wordlists from personal data, so the intended approach was to build a targeted dictionary from the victim's information and then verify candidates against the SHA-1 hash.

3. Tools and Commands Used

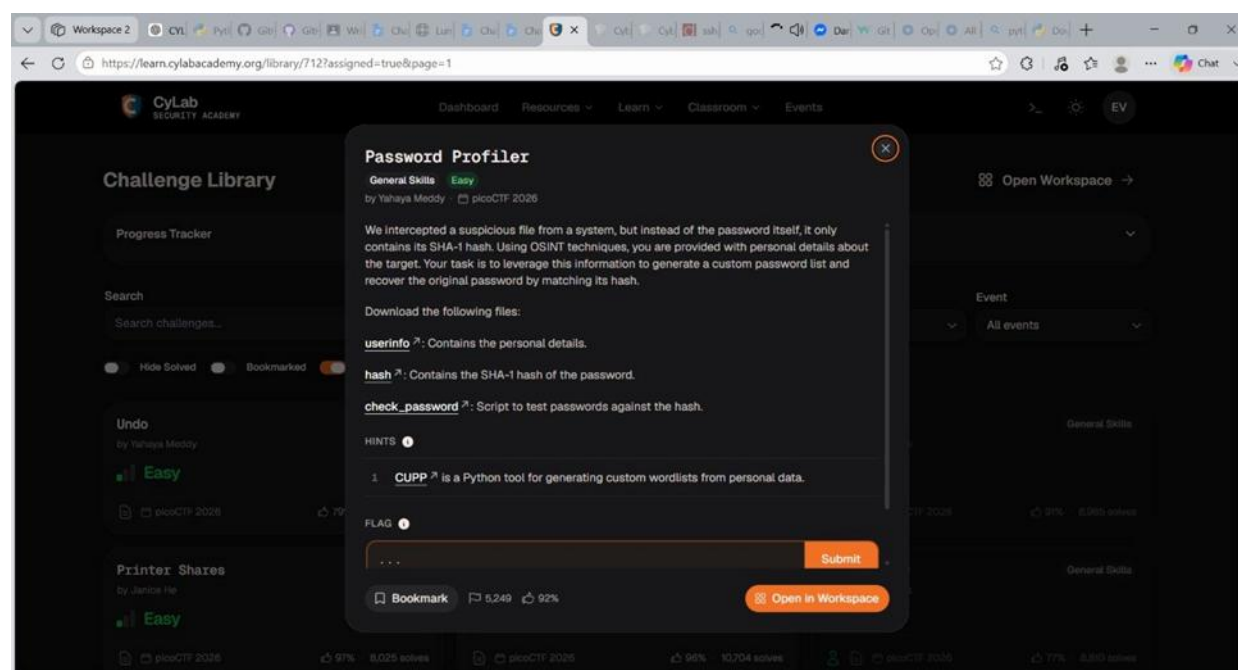
Tool / Command	What it was used for	Result / Output
cupp -i	Generate a custom password list from the victim's personal information.	CUPP generated alice.txt with 180 candidate words.
ls	Check the files available in the workspace.	Confirmed the challenge files and generated wordlist were present.
cat userinfo.txt	Read the target's personal details.	Alice Johnson, nickname AJ, birthdate 15-07-1990, partner Bob, and child Charlie.
cat hash.txt	Read the SHA-1 value to match.	968c234940273dd57dc4be7e238c5ac200ceac5.

Tool / Command	What it was used for	Result / Output
check_password.py	Understand and verify the password check.	The script hashes each candidate with SHA-1 and compares it with the supplied hash.
Python hashlib	Compare wordlist candidates against the SHA-1 hash.	The matching candidate was AJ_15901990.

4. Solution Walkthrough

Step 1

I opened the Password Profiler challenge and revealed the available hint first. The hint identified CUPP as a Python tool for generating custom wordlists from personal data. This established the intended direction for the solution.



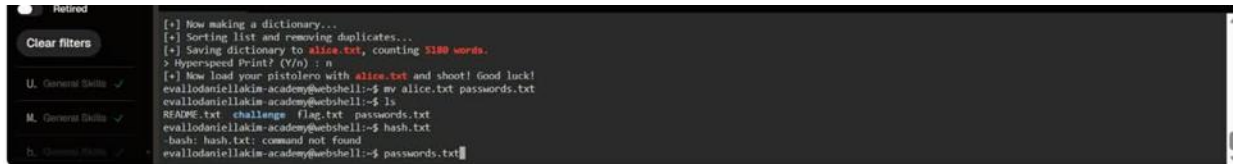
Step 2

I reviewed the files available in the workspace and confirmed that the challenge supplied userinfo.txt, hash.txt, and check_password.py. I also tested the available files and commands in the terminal before generating the custom list.



Step 3

Following the hint, I ran `cupp -i`. **CUPP** asked for personal details about the target. I entered the information shown in the screenshots and left the optional keyword, special-character, random-number, and leet-mode options disabled.



```
[+] Now making a dictionary...
[+] Sorting list and removing duplicates...
[+] Saving dictionary to alice.txt, counting 5180 words.
> Hyperspeed Print? (Y/n) : n
[+] Now load your pistolero with alice.txt and shoot! Good luck!
evallodaniellakin-academy@ubuntu:~$ cat alice.txt
passwords.txt
challenge.txt
flag.txt
README.txt
evallodaniellakin-academy@ubuntu:~$ ls
README.txt  challenge  flag.txt  passwords.txt
evallodaniellakin-academy@ubuntu:~$ cat hash.txt
-bash: hash.txt: command not found
evallodaniellakin-academy@ubuntu:~$ cat passwords.txt
```

Step 4

It shows CUPP generating the custom dictionary. I ran ``cupp -i``, entered the target's profile information, and CUPP generated 180 candidate words for the password search.



```
> Do you want to add some key words about the victim? Y/[N]: N
> Do you want to add special chars at the end of words? Y/[N]: N
> Do you want to add some random numbers at the end of words? Y/[N]: N
> Leet mode? (i.e. leet = 1337) Y/[N]: N

[+] Now making a dictionary...
[+] Sorting list and removing duplicates...
[+] Saving dictionary to alice.txt, counting 5180 words.
> Hyperspeed Print? (Y/n) : N
```

STEP 5

Entered the target's basic personal details, such as the first name, surname, and nickname, so CUPP could generate likely password combinations.



```

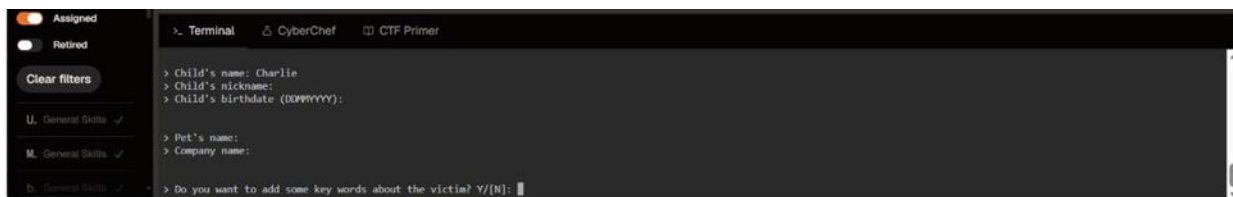
  \      /
  (oo)\_/_
    _/  _/
   /_  _/
  /_  _/
 /_  _/
/_  _/

# passwords
# profiler
#
[ Maris Kargas | j0rged@remote-exploit.org ]
[ Mebus | https://github.com/Mebus/ ]

[+] Insert the information about the victim to make a dictionary
[+] If you don't know all the info, just hit enter when asked! ;)
> First Name: Charlie
```

Step 6

The CUPP asking for the child's information. I entered the provided child information because family-related details can also be included in the generated candidate list.



```
> Child's name: Charlie
> Child's nickname:
> Child's birthdate (DDMMYYYY): D0M3Y0000

> Pet's name:
> Company name:

> Do you want to add some key words about the victim? Y/[N]:
```

Step 7

I continued entering the available information and left the optional keyword, special-character, random-number, and leet-mode options disabled, as described in the challenge walkthrough.



Step 8

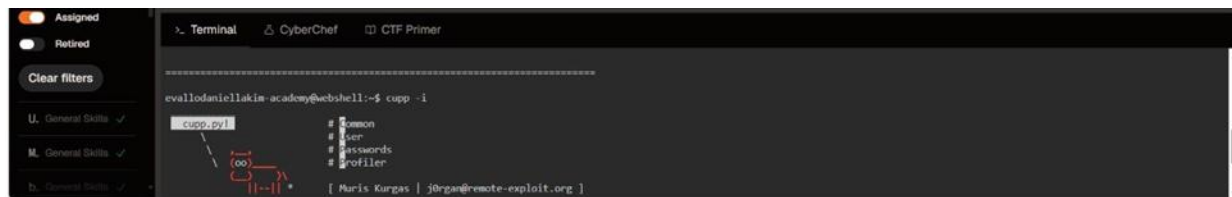
It shows the completed basic profile information entered into CUPP, including the first name, surname, nickname, and birthdate. CUPP uses these values to build the custom password dictionary.



STEP 9

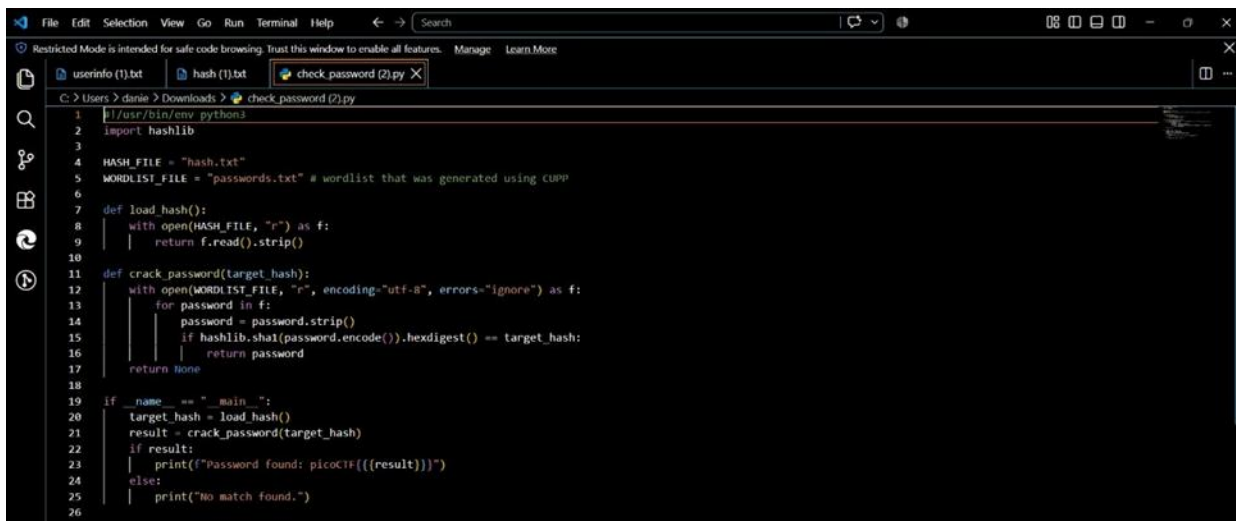
TheCUPP starting in the terminal with the Password Profiler banner. The ``cupp -i`` command launched the interactive mode used to collect the target information and generate the custom wordlist.

After the profile was completed, CUPP generated a custom dictionary. The generated list contained 180 words and was saved as `alice.txt`; it was then used as the candidate password list.

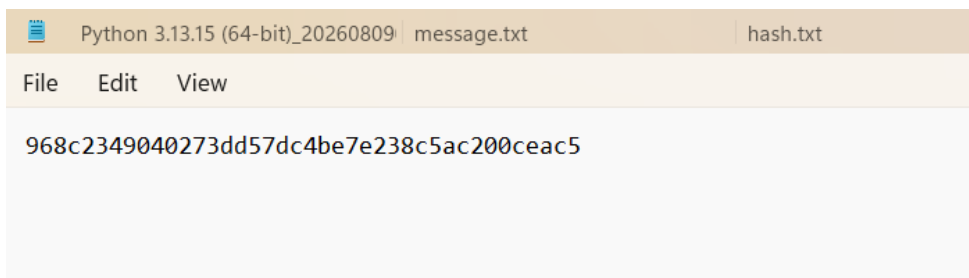


STEP 10

I inspected the provided Python checker to understand exactly how candidates were verified. The script reads `hash.txt`, loops through `passwords.txt`, computes SHA-1 for each candidate, and prints the flag when a candidate matches the target hash.



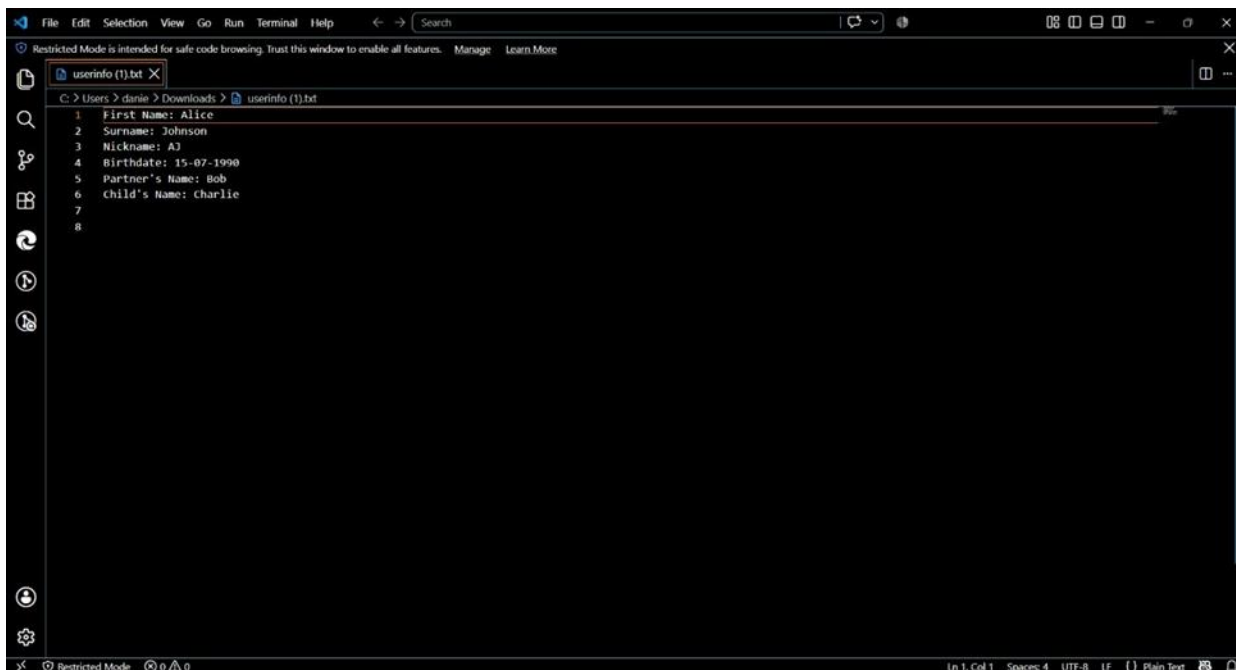
```
1 #!/usr/bin/env python3
2 import hashlib
3
4 HASH_FILE = "hash.txt"
5 WORDLIST_FILE = "passwords.txt" # wordlist that was generated using CUPP
6
7 def load_hash():
8     with open(HASH_FILE, "r") as f:
9         return f.read().strip()
10
11 def crack_password(target_hash):
12     with open(WORDLIST_FILE, "r", encoding="utf-8", errors="ignore") as f:
13         for password in f:
14             password = password.strip()
15             if hashlib.sha1(password.encode()).hexdigest() == target_hash:
16                 return password
17     return None
18
19 if __name__ == "__main__":
20     target_hash = load_hash()
21     result = crack_password(target_hash)
22     if result:
23         print(f"Password found: picoCTF{{{result}}}")
24     else:
25         print("No match found.")
26
```



Python 3.13.15 (64-bit)_20260809 | message.txt | hash.txt

File Edit View

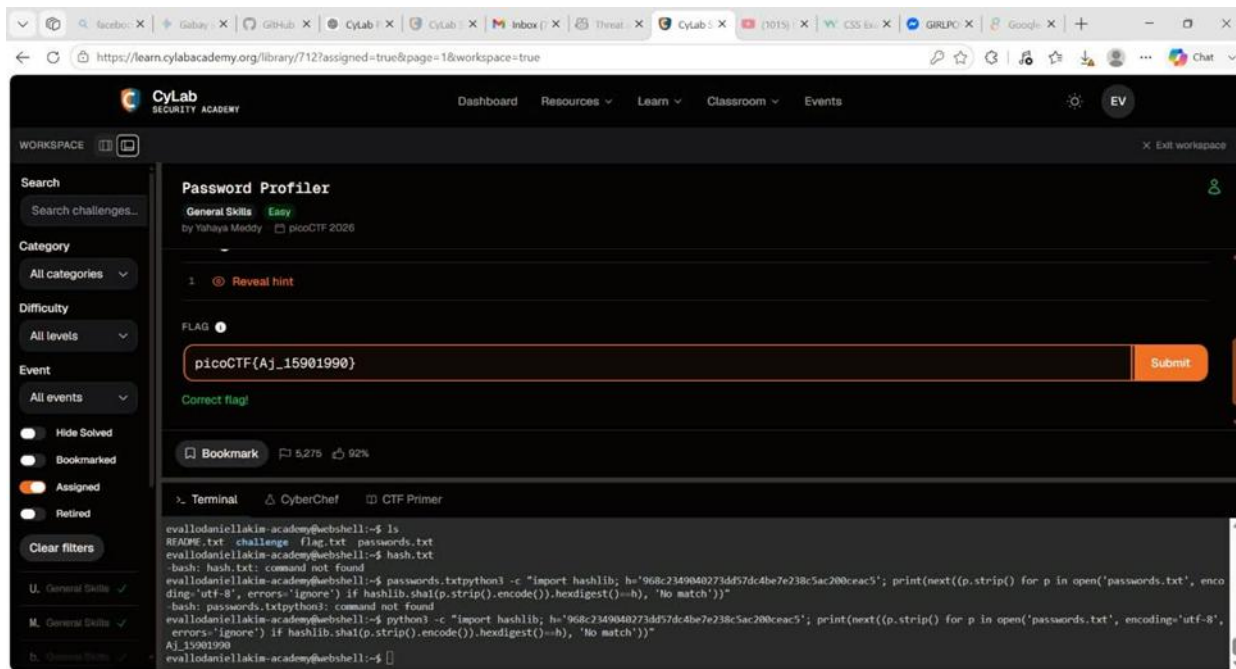
968c2349040273dd57dc4be7e238c5ac200ceac5



```
1 First Name: Alice
2 Surname: Johnson
3 Nickname: AJ
4 Birthdate: 15-07-1990
5 Partner's Name: Bob
6 Child's Name: Charlie
7
8
```

Step 11

I checked the generated candidates against the supplied SHA-1 hash. The matching password was AJ_15901990. The challenge accepted the corresponding flag.



5. Flag

picoCTF{Aj_15901990}

6. Key Concept / What I Learned

This challenge demonstrated password profiling and targeted wordlist generation. Personal details such as names, nicknames, dates, and family information can be transformed into likely password candidates. A known SHA-1 hash can then be used to verify the correct candidate. The main lesson is that predictable personal information can make passwords easier to recover.

7. References

YouTube – picoCTF 2026 / CyLab - Password Profiler

<https://youtu.be/Qqe9tmHHeI?si=8oQzkvn7KVU1WIVH>, [picoCTF 2026 | Denton's Blog](#)